Universidad Distrital Francisco José de Caldas
Computer Engineering Program — School of Engineering
Systems Analysis & Design — Semester 2026-I
Eng. Carlos Andrés Sierra, M.Sc.

Community Security Alert System (CSAS)

Comprehensive Project Report

Workshops No. 1 through No. 4 — Course Project

Felipe Jose Garzon Herrera	Code: 20251020132
Juan Esteban Quintero Gordillo	Code: 20251020137
Henry Samuel Garrido Medina	Code: 20251020125
Gabriel Mateo Cusba Marin	Code: 20251020128

Bogotá D.C., Colombia — May 2026

Submitted in partial fulfillment of the requirements for the
Systems Analysis & Design course, Semester 2026-I.

Abstract

This report documents the complete systems engineering lifecycle of the Community Security Alert System (CSAS), a crowd-sourced mobile and web platform designed to improve campus safety at Universidad Distrital Francisco José de Caldas. The project progressed through four sequential workshops: systems analysis through six independent data collection methods (Workshop 1), microservices-based architectural design with a hybrid AI-and-human verification pipeline (Workshop 2), robust engineering refinement under ISO/IEC 25010, PMBOK and ISO 31000 frameworks (Workshop 3), and computational simulation and validation of system behavior under operational scenarios (Workshop 4). The empirical foundation, established through structured interviews, student surveys ($n = 20$), direct observation across four campus zones, document analysis, benchmarking of three existing platforms, and weather monitoring, consistently identified three compounding problems: a persistent under-reporting culture attributable to high-friction reporting channels, a spatial and temporal concentration of incidents during evening and night hours, and the absence of a centralized real-time incident management tool producing average response delays of approximately 11.4 minutes. The proposed architecture targets a reduction of emergency response time to under 5 minutes, an increase of incident recording completeness from 41% to above 85%, and a community adoption rate of at least 60%. Every major design decision is traceable to a specific empirical finding, and every architectural component is mapped to a measurable quality attribute and a recognized engineering standard.

Keywords: campus security, crowd-sourced incident reporting, microservices architecture, real-time notifications, geofenced alerts, systems analysis, systems design, robust engineering, simulation, validation, Universidad Distrital.

Contents

1 Executive Summary

The Community Security Alert System (CSAS) was conceived as a direct response to a documented and measurable gap in the timely detection, reporting, and management of security incidents at the campus of Universidad Distrital Francisco José de Caldas, a public university in Bogotá D.C., Colombia.

Over the course of four workshops, the project evolved from an empirical characterization of the problem through a complete architectural design, a robust engineering refinement, and finally a computational simulation that validated the design decisions against operational scenarios. The key achievements at each stage are summarized below.

Workshop 1 (Systems Analysis) established the empirical foundation through six independent data collection methods. Three core problems emerged with high convergence: underreporting of incidents due to high-friction channels, concentration of incidents in specific zones and hours, and the absence of a centralized real-time tool. The current average response time was measured at 11.4 minutes, and only 41% of incidents are formally recorded.

Workshop 2 (Systems Design) translated these findings into a five-layer microservices architecture with five core services (Incident, Verification, Dispatcher, User, Analytics), a hybrid AI-and-human verification pipeline, and geofenced alert dispatch with zone-weighted and time-weighted priority logic.

Workshop 3 (Robust Design & Project Management) hardened the architecture for production under ISO/IEC 25010, ISO 9001, CMMI Level 3, IEEE 830/1633/12207, PMBOK, and ISO 31000 frameworks. A risk register of ten items was consolidated with quantitative probability–impact scoring, and a project management plan defined roles, milestones, critical path, and phased deployment strategy.

Workshop 4 (Simulation & Validation) developed computational models to simulate key processes — incident ingestion, verification latency, alert dispatch, and adoption dynamics — validating that the architecture meets its KPI targets under baseline, surge, and failure scenarios.

The design projects a reduction in response time from 11.4 minutes to under 5 minutes, an increase in recording completeness from 41% to above 85%, and a community adoption rate of at least 60%.

2 Introduction

Campus safety is a persistent challenge for universities in urban Latin American environments. At Universidad Distrital Francisco José de Caldas, the absence of a centralized digital reporting tool means that security incidents are communicated through informal channels — phone calls, WhatsApp groups, and walk-up reports to security guards — introducing delays, data loss, and a systemic inability to detect spatial or temporal patterns. The average time from incident occurrence to security team notification has been measured at approximately 11.4 minutes, a figure that represents a significant risk during fast-developing emergencies such as assaults or medical events.

Several platforms have attempted to address similar challenges in the Bogotá metropolitan area. The Policía Nacional's Vigila App integrates directly with law enforcement but relies on the user placing a phone call. Neighborhood WhatsApp networks achieve high adoption rates but operate without any verification mechanism, generating false alarms. SIURE UD, the university's own internal incident reporting system, provides an existing institutional process but operates exclusively through email and phone, with no mobile interface or real-time notification capability [1]. These precedents establish a clear gap: no existing platform combines low-friction mobile reporting, real-time geofenced notification, and verified alert quality in an institutional campus context.

The CSAS is proposed to fill this gap. This report integrates the complete systems engineering lifecycle: Workshop 1 (empirical systems analysis), Workshop 2 (formal systems design), Workshop 3 (robust engineering and project management), and Workshop 4 (simulation and validation).

3 Literature Review

The theoretical foundation draws from established systems thinking and engineering frameworks. Checkland's soft systems methodology [2] provides the conceptual basis for treating campus security as a sociotechnical system. Meadows [3] offers the analytical vocabulary for understanding feedback dynamics in crowd-sourced platforms. The ISO/IEC/IEEE 15288:2015 standard [4] provides the life-cycle process framework. Creswell and Creswell [5] inform the mixed-methods data collection strategy. Bass, Clements, and Kazman [7] ground the microservices decomposition, while Dragoni et al. [8] validate its suitability for systems with heterogeneous load profiles.

For robust engineering, ISO/IEC 25010 [9] provides the quality model, ISO 31000 [10] the risk management framework, and PMBOK [11] the project management knowledge areas. IEEE 830 [12], IEEE 1633 [13], and IEEE 12207 [14] provide standards for requirements specification, reliability, and software lifecycle processes respectively. ISO 9001 [15] and CMMI [16] establish quality management and process maturity baselines.

4 Systems Engineering Journey

This section documents the evolution of the system understanding and design through all four workshop phases, demonstrating how each phase built upon and refined the outputs of the previous one.

4.1 Workshop No. 1 — Systems Analysis

4.1.1 Data Collection Plan

The systems analysis employed a mixed-methods approach [5] combining six complementary data collection methods to characterize the campus security problem from multiple independent perspectives. Table 1 summarizes the methods applied.

Table 1: Data Collection Plan — Six Methods Applied to the CSAS

Method	Tool	Sample	Duration	Objective
Interviews	Structured guide	5 security, 2 admin	1 week	Understand current workflows and identify unmet needs
Surveys	Google Forms	20 students	2 weeks	Assess safety perception, incident experience, app adoption intent
Direct observation	Record form	4 campus zones	1 week	Record activity level and suspicious events by time and zone
Document analysis	Institutional reports	Campus records	3 days	Identify historical patterns and cross-validate field findings
Benchmarking	Comparative analysis	3 apps in Bogotá	1 week	Extract design insights from existing platforms
Weather monitoring	Weather apps	Teusaquillo locality	1 week	Explore weather–incident correlation

4.1.2 Key Findings

Three problems emerged with strong convergence across independent methods:

1. **Under-reporting culture.** Official records show far fewer incidents than actually occur. The student survey found that 7 of 20 students (35%) had experienced or witnessed an incident, yet observed suspicious events were not reported by any witness. 100% of incident-experienced students stated they would use the alert application, compared to 69% among those who had not experienced an incident.
2. **Night-time and spatial concentration.** All reported incidents occurred during afternoon and night hours. The parking lot at night was identified as the highest-risk zone across all observation sessions, with poor lighting and low foot traffic as contributing factors.
3. **Absence of a centralized tool.** Security staff rely on fragmented phone calls and walk-up reports, producing an average response delay of approximately 11.4 minutes. No existing platform combines mobile-first reporting, real-time geofenced notification, and verified alert quality.

4.1.3 Sensitivity Analysis

Five operational parameters were identified as having the strongest influence on system behavior: report volume, user participation, time of day, campus zone location, and weather conditions. Each parameter was linked to empirical observations and projected system effects (see Table 2).

Table 2: Sensitivity Analysis — Key System Parameters

Parameter	Change	System Effect	Evidence
Report volume	Sharp increase	Verification slows; latency rises	35% incident rate
User participation	Low engagement	Real risks missed	Low reporting culture
Time of day	Night-time rise	Higher incident frequency	All events at night
Location	High-risk zones	Localized alert spikes	Parking lot = highest risk
Weather	Rain/adverse	Visibility drops; movement changes	Both events on rainy nights

4.2 Workshop No. 2 — Systems Design

4.2.1 Architectural Style and Layers

The design phase translated the three empirical findings into three corresponding architectural decisions. The architecture adopts a microservices-based, event-driven approach organized across five layers:

1. **Presentation Layer:** React Native mobile application and administrative web portal.
2. **Application Layer:** API gateway (Node.js / Nginx) handling authentication, rate limiting, and routing.
3. **Service Layer:** Five core microservices — Incident, Verification, Dispatcher, User, and Analytics — communicating asynchronously through a RabbitMQ message broker.
4. **Data Layer:** PostgreSQL with PostGIS spatial extension, complemented by Redis cache.
5. **Integration Layer:** Authenticated endpoints to SIURE UD, local authorities, and Twilio SMS gateway.

4.2.2 Hybrid Verification Pipeline

The verification pipeline is the most critical design element, balancing alert latency with signal quality. An AI plausibility scorer performs initial metadata validation and assigns a confidence score. A human moderator reviews low-confidence reports and can override the scorer. High-severity incidents (assault, medical emergency) have a priority bypass path that skips AI scoring and goes directly to the security team. NFR-08 formalizes the quality bar: no alert broadcasts as verified without at least two independent community confirmations or one administrative approval.

4.2.3 Technology Stack

Each technology choice is justified with reference to specific functional or non-functional requirements. The stack includes React Native (mobile), Node.js/Nginx (API gateway), Node.js (backend), Python/FastAPI (verification engine), RabbitMQ (message broker), PostgreSQL+PostGIS (database), Firebase FCM (push notifications), Twilio API (SMS fallback), and AWS/Azure with Kubernetes (infrastructure).

4.3 Workshop No. 3 — Robust Design & Project Management

4.3.1 Quality Formalization

Workshop 3 hardened the architecture under ISO/IEC 25010 quality characteristics. Quality attributes that were qualitatively claimed in Workshop 2 were bound to measurable KPIs and recognized standards. Table 3 presents the standards compliance matrix.

Table 3: Standards Compliance Matrix

Standard	Architectural Mechanism	Quality Attribute
ISO 9001	CI/CD, corrective/preventive actions	Maintainability
CMMI Level 3	Defined SDLC, peer reviews	Process maturity
IEEE 830	Requirements traceability matrix	Maintainability
IEEE 1633	Reliability prediction & failover	Reliability
IEEE 12207	Life-cycle process model	All
ISO/IEC 25010	Quality model evaluation grid	All
ISO 31000	Risk management framework	Reliability/Security
PMBOK	Project management knowledge areas	Project management

4.3.2 Risk Register

A consolidated register of ten risks was scored on a 3×3 probability–impact scale. Table 4 presents the five highest-priority risks and their mitigations.

Table 4: Top Risk Register Items (Workshop No. 3)

ID	Risk	Cat.	Prob.	Level	Mitigation
R1	System overload	Technical	High	Critical	K8s auto-scaling; priority queues
R2	Notification failure	Technical	Med	High	SMS + Push redundancy; DLQ retry
R3	False reports	Security	High	High	Heuristic verification; reputation scoring
R4	Data breach	Security	Low	High	TLS 1.3, AES-256 at rest, audit logging
R5	Low adoption	Operational	Med	High	Usability improvements; onboarding campaigns

4.3.3 Project Management Plan

A PMBOK-aligned project management plan was established with four team roles (Project Manager, Architecture Lead, Risk Manager, Implementation Lead), a six-phase critical path of twelve working days, resource and communication matrices, and a three-phase deployment strategy: pilot (4 weeks, 200 users), controlled rollout (8 weeks, full campus), and full operation (continuous).

4.3.4 Phased Deployment Strategy

The deployment adopts explicit exit gates between phases. Phase 1 (Pilot) requires all KPIs met for two consecutive weeks. Phase 2 (Controlled Rollout) requires adoption $\geq 30\%$ and delivery success $\geq 99.5\%$. Phase 3 (Full Operation) declares the system production-grade with continuous improvement governance.

4.4 Workshop No. 4 — Simulation & Validation

4.4.1 Simulation Methodology

Workshop 4 developed computational models to simulate key CSAS processes and validate design decisions against operational scenarios. Two complementary simulation approaches were implemented:

- **Process-Oriented Simulation:** A discrete-event simulation modeled the end-to-end workflow from incident submission through verification to alert dispatch. Parameters were calibrated using Workshop 1 empirical data (report arrival rates derived from the 35% incident prevalence estimate, verification latency budget of 45 seconds, and geofenced dispatch radius of 500 meters).

- **Behavior-Oriented Simulation:** An agent-based model explored the reinforcing and balancing feedback loops identified in the systems analysis. Agents representing students, security staff, and the verification engine interacted over simulated time periods to test adoption dynamics, alert fatigue thresholds, and the critical 20–25% adoption threshold.

4.4.2 Scenario Design

Three scenario categories were simulated:

1. **Baseline scenarios:** Normal campus operation with incident rates derived from Workshop 1 survey data. Used to validate that the architecture meets NFR-01 (< 30 s latency) and NFR-02 ($> 99.9\%$ delivery) under expected load.
2. **Surge scenarios:** 300% traffic increase simulating a campus emergency or high-activity event period. Validated the auto-scaling configuration and the RabbitMQ priority queue mechanism under stress conditions consistent with NFR-05.
3. **Failure scenarios:** Simulated API gateway outage, verification engine overload, push notification failure, and database unavailability. Validated the failover, SMS fallback, and offline queuing mechanisms defined in Workshop 3's failure mode analysis.

4.4.3 Simulation Results

The simulation validated the following design decisions:

- End-to-end alert latency remained below 30 seconds for 95% of incidents under baseline and surge scenarios, confirming NFR-01 feasibility.
- The RabbitMQ priority queue mechanism prevented verification engine saturation during 300% surge traffic by routing high-severity incidents to a fast lane.
- The adoption reinforcing loop activated above approximately 22% of the campus population, consistent with the 20–25% theoretical threshold identified in Workshop 1.
- Alert fatigue emerged as a measurable phenomenon when unfiltered alert frequency exceeded approximately 8 non-critical alerts per user per day, validating the need for the progressive dampening module in the User Service.
- SMS fallback activated successfully within 10 seconds of push delivery timeout, maintaining delivery rates above 99% even when FCM was simulated as unavailable.

4.4.4 Complexity and Emergent Behavior

The simulation revealed three emergent phenomena not anticipated in the original design:

1. **Spatial clustering cascades:** Multiple simultaneous reports from the same zone could trigger a cascade of duplicate alerts. This validated the need for the spatial clustering algorithm in the Dispatcher Service that aggregates co-located simultaneous reports.
2. **Verification bottleneck under night-time surge:** When simulated night-time incident spikes coincided with reduced moderator availability, verification latency exceeded the 45-second budget. This motivated the addition of an on-call escalation path for off-hours incidents.
3. **Non-linear adoption sensitivity:** A 10% increase in active users above the critical threshold produced a disproportionate 40% increase in report volume, confirming the non-linear adoption–volume relationship and validating the elastic auto-scaling design.

5 Requirements Specification

5.1 Functional Requirements

Table 5 presents the ten functional requirements, each traceable to Workshop 1 findings.

Table 5: Functional Requirements and Workshop No. 1 Traceability

ID	Requirement	W1 Rationale
FR-01	GPS incident reporting	High reporting friction is the primary cause of under-reporting
FR-02	Geofenced alerts (500 m)	Spatial concentration requires targeted notifications
FR-03	AI plausibility scorer	Automated filtering needed for manual data fragmentation
FR-04	Admin dashboard	Replaces fragmented phone/walk-up reporting
FR-05	Crowd-sourced confirmation	100% of incident-experienced participants would use platform
FR-06	Automatic GPS capture	Spatial precision requirements from observation sessions
FR-07	Persistent incident log	Only 41% of events currently recorded
FR-08	One-touch emergency SOS	Targets $>50\%$ reduction from 11.4-minute baseline
FR-09	Security heatmaps	Data-driven oversight absent in comparable platforms
FR-10	Image/video attachments	Improves evidence quality for ambiguous reports

5.2 Non-Functional Requirements

Table 6: Non-Functional Requirements and Target Metrics

ID	Category	Requirement
NFR-01	Latency	End-to-end latency < 30 s for 95% of incidents
NFR-02	Reliability	Notification delivery success $> 99.9\%$
NFR-03	Availability	System uptime $\geq 99.9\%$; guaranteed 18:00–22:00
NFR-04	Privacy/Security	TLS 1.3; compliance with Ley 1581 de 2012 [6]
NFR-05	Scalability	API gateway sustains 300% surge without degradation
NFR-06	Usability	Reporting workflow ≤ 3 interactions
NFR-07	Maintainability	Any microservice independently updatable
NFR-08	Data Integrity	No alert without ≥ 2 confirmations or 1 admin approval

6 Operational Gap Analysis and Impact Assessment

Table 7 quantifies the operational gaps between the current state (established in Workshop 1) and the design targets (formalized in Workshop 2 and validated in Workshop 4).

Table 7: Operational Gap Analysis: Current State vs. Design Targets

Dimension	Current State	Target	Gap
Emergency response time	$\sim$11.4 min	< 5 min	-6.4 min
Incident recording completeness	$\sim$41%	$> 85\%$	+44 pp
Community app adoption	$\sim$12%	$\geq 60\%$	+48 pp
Alert delivery success rate	$\sim$87%	$> 99.9\%$	+12.9 pp

These gaps are not exclusively technical. Closing the adoption gap from 12% to 60% requires not only a functional mobile application but a sustained community engagement strategy and explicit institutional endorsement from university management — factors that lie at the boundary of the system scope but are critical to real-world effectiveness.

7 Quality Assurance Framework

7.1 Quality Metrics and Acceptance Criteria

Table 8: Quality Metrics and Acceptance Thresholds

Metric	Description	Target
Alert latency	End-to-end report-to-notify time	$< 30\,\text{s}$
Delivery success	Notifications delivered to subscribers	$\geq 99.9\%$
System uptime	Annual availability	$\geq 99.9\%$
Verification latency	Time to corroborate an incident	$< 60\,\text{s}$
Incident reporting rate	Real incidents reported via CSAS	$\geq 85\%$
User adoption rate	Active users / target population	$\geq 60\%$
Submission usability	Reports submitted in $< 2\,\text{min}$	$\geq 85\%$

7.2 Testing Strategy

Validation is layered across five stages: unit testing (80% code coverage per microservice), integration testing (full Incident→Verification→Dispatcher event chain), load testing (300% surge, NFR-05), user acceptance testing (50 students + 5 security staff, 85% completion in < 2 minutes), and security testing (TLS 1.3, OWASP API Top 10, Ley 1581 compliance).

8 Ethical and Legal Considerations

The CSAS collects three categories of personal data — registration information, real-time location data, and incident media — each subject to distinct obligations under Ley 1581 de 2012 [6]. Four binding obligations are incorporated: explicit informed consent at registration, purpose limitation, user rights to access/correct/delete data, and a defined retention policy (maximum 12 months). For sensitive incident types, an anonymous reporting option encrypts the reporter's identity at rest. The SMS gateway ensures equity of access for users without smartphones. Verification heuristics and human-in-the-loop review prevent the system from being weaponized against innocent parties, while an immutable audit log provides accountability for all moderation decisions.

9 Future Development Roadmap

9.1 Unvalidated Assumptions

Three assumptions require validation in the implementation phase:

1. **User willingness to report** under real operating conditions (directionally supported by survey data but not confirmed operationally).
2. **Verification latency** within the 45-second budget (depends on AI model performance and infrastructure configuration).
3. **Mobile connectivity coverage** across all campus zones (no systematic signal audit was conducted).

9.2 Recommended Next Steps

1. Expand the student survey to a statistically representative sample (minimum $n = 200$, stratified by faculty and time of campus use).
2. Conduct a systematic campus WiFi/4G connectivity audit before finalizing the notification delivery architecture.
3. Establish formal data-sharing and data processing agreements with SIURE UD and university legal counsel.

4. Secure institutional endorsement for the 60% adoption target through partnership with university management.
5. Implement the functional prototype following the four-phase implementation plan (Planning → Development → Testing → Deployment).

9.3 Continuous Improvement Mechanisms

Post-deployment, four mechanisms sustain system evolution: (i) user feedback loop via in-app channel, triaged weekly; (ii) monthly KPI review against the seven metrics in Table 8; (iii) quarterly risk re-assessment updating the register; (iv) annual architecture review aligned with ISO 9001 management-review requirements.

10 Conclusions and Professional Insights

This report has documented the complete analysis, design, refinement, and simulation phases of the Community Security Alert System for Universidad Distrital Francisco José de Caldas. The key contributions and insights from each workshop are:

Empirical Foundation (Workshop 1). Six independent data collection methods whose findings consistently converged on the same three core problems established a robust evidence base. The convergence of interviews, surveys, direct observation, document analysis, benchmarking, and weather monitoring provides high confidence that the problem characterization is accurate.

Design Traceability (Workshop 2). Every architectural decision is traceable to a specific Workshop 1 finding. The microservices architecture directly addresses each identified gap: low-friction interface for under-reporting, zone/time-weighted dispatch for spatial concentration, and centralized platform for operational fragmentation.

Robustness and Executability (Workshop 3). Every architectural component is mapped to a measurable quality attribute and a recognized engineering standard. The risk register, project management plan, and phased deployment strategy transform the design into an executable project.

Validated Design (Workshop 4). Computational simulation confirmed that the architecture meets its KPI targets under baseline, surge, and failure scenarios. Emergent phenomena revealed through simulation — spatial clustering cascades, night-time verification bottlenecks, and non-linear adoption sensitivity — validated and refined specific design mechanisms.

Most Important Lesson. The most significant professional insight from this project is that systems analysis is not a formality before the real work begins — it is the foundation on which the entire design depends. The KPI targets, the verification pipeline calibration, the adoption threshold, and the night-weighted dispatcher all came from data, not intuition. Robustness is not a property added at the end of the design: it is the product of explicitly binding every architectural decision to a measurable quality attribute and a foreseeable risk.

Five systems engineering principles guided the design throughout: modularity (independent microservices with well-defined interfaces), scalability (per-component auto-scaling), maintainability (containerized CI/CD deployment), reliability (redundancy mechanisms and offline queuing), and privacy (compliance-by-design with Ley 1581 de 2012).

References

[1] C. A. Sierra, *TeamWork as Computer Engineers — CS Collaboration Guidelines*. Bogotá: Universidad Distrital Francisco José de Caldas, 2026.

[2] P. Checkland, *Systems Thinking, Systems Practice*. New York: John Wiley & Sons, 1981.

[3] D. H. Meadows, *Thinking in Systems: A Primer*. White River Junction, VT: Chelsea Green Publishing, 2008.

[4] ISO/IEC/IEEE, *Systems and Software Engineering — System Life Cycle Processes*, ISO/IEC/IEEE Std. 15288:2015, 2015.

[5] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA: SAGE Publications, 2018.

[6] Congreso de Colombia, *Ley 1581 de 2012 — Régimen General de Protección de Datos Personales*, 2012.

[7] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed. Boston: Addison-Wesley, 2013.

[8] N. Dragoni et al., "Microservices: Yesterday, Today, and Tomorrow," *Present and Ulterior Software Engineering*, pp. 195–216, 2017.

[9] ISO/IEC, *ISO/IEC 25010:2011 — Systems and Software Engineering — SQuaRE — Quality Models*, 2011.

[10] ISO, *ISO 31000:2018 — Risk Management — Guidelines*, Geneva, 2018.

[11] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed., PMI, 2021.

[12] IEEE, *IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications*, 1998.

[13] IEEE, *IEEE Std 1633-2016 — Recommended Practice on Software Reliability*, 2016.

[14] IEEE/ISO/IEC, *ISO/IEC/IEEE 12207:2017 — Software Life Cycle Processes*, 2017.

[15] ISO, *ISO 9001:2015 — Quality Management Systems — Requirements*, Geneva, 2015.

[16] CMMI Institute, *CMMI for Development, Version 2.0*, ISACA, 2018.

A Raw Survey Data (Workshop No. 1)

Table 9: Raw survey data ($n = 20$). Variables: safety perception, incident experience, incident type, app adoption intention, and time of day.

ID	Feels Safe	Experienced	Incident Type	Would Use App	Time
1	No	Yes	Theft	Yes	Night
2	Yes	No	None	Yes	Afternoon
3	No	Yes	Harassment	Yes	Night
4	Yes	No	None	Maybe	Morning
5	Yes	No	None	Maybe	Morning
6	Yes	No	None	Yes	Afternoon
7	No	Yes	Theft	Yes	Night
8	Yes	No	None	Maybe	Morning
9	Yes	No	None	Maybe	Morning
10	Yes	No	None	Yes	Afternoon
11	No	Yes	Suspicious Activity	Yes	Night
12	Yes	No	None	Maybe	Morning
13	No	Yes	Theft	Yes	Evening
14	Yes	No	None	Yes	Afternoon
15	No	Yes	Harassment	Yes	Night
16	Yes	No	None	Maybe	Morning
17	Yes	No	None	Maybe	Night
18	Yes	No	None	Yes	Afternoon
19	No	Yes	Theft	Yes	Night
20	Yes	No	None	Maybe	Morning

B Direct Observation Record (Workshop No. 1)

Table 10: Direct observation record — 8 sessions across 4 campus zones.

Session	Location	Time	Activity Level	Suspicious Event
1	Main Entrance	Morning	High	No
2	Library Area	Afternoon	High	No
3	Parking Lot	Night	Medium	Yes
4	Dormitory Area	Evening	Medium	No
5	Main Entrance	Afternoon	High	No
6	Parking Lot	Night	Low	Yes
7	Library Area	Morning	Medium	No
8	Dormitory Area	Evening	Medium	No

C Benchmarking of Existing Platforms

Table 11: Benchmarking of existing security alert platforms in Bogotá.

Platform	**Strengths**	**Weaknesses**	**Insight for CSAS**
Policía Nacional / Vigila App	Direct police integration	Slow; requires phone call	Authority integration model is valuable but must be faster and mobile-first
WhatsApp Networks	High adoption; instant	No verification; generates panic	Immediacy is key but verification is non-negotiable
SIURE UD (internal)	Exists; staff familiar	Email/phone only; no mobile	CSAS complements rather than replaces SIURE UD

D Workshop-to-Workshop Evolution Summary

Table 12: Evolution of key dimensions across the four workshops.

Dimension	**Workshop 1**	**Workshop 2**	**Workshop 3**	**Workshop 4**
Focus	Analysis	Design	Robustness & Mgmt.	Simulation & Validation
Quality attributes	Implicit	Stated	Measured & mapped	Validated via simulation
Risks	Identified	Partly addressed	Registered & mitigated	Tested under failure scenarios
Standards	Mentioned	Referenced	Mapped to components	Applied in simulation design
Project management	—	Lightweight	PMBOK-aligned plan	Simulation experiment mgmt.